DoS/DDoS - TCP SYN Flooding Attack and Defense

Many Internet applications like the Web services are running over TCP at the transport layer which provides reliable communications by establishing connection using the infamous 3-way handshake as shown in Figure 1 prior to commencing communications.

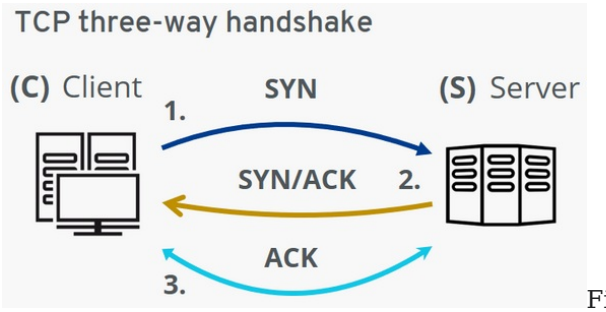

Figure 1

However, what if an attacker were to misuse the TCP 3-way handshake by sending many SYN requests with spoofed source IP addresses as shown in Figure 2?

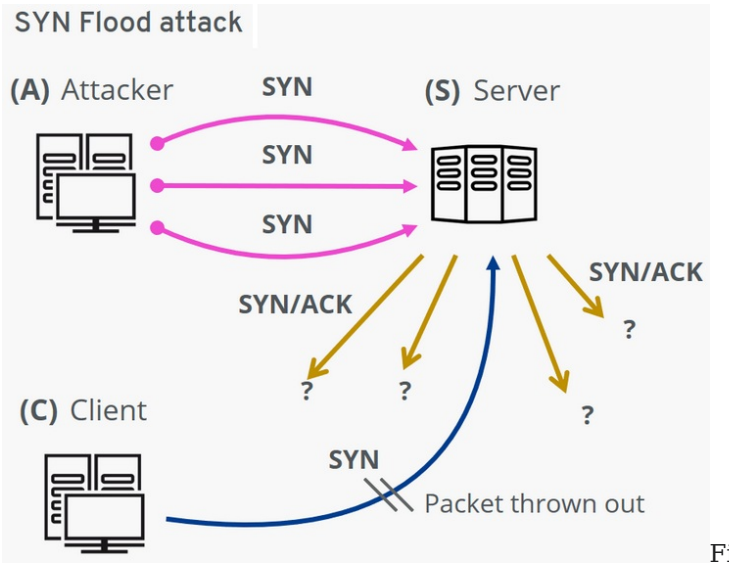

Figure 2

The SYN/ACK sent out by the server to the spoofed IP addresses will not receive corresponding ACK and will result in many half-open or embryonic connections which will use up the resources of the server. Eventually, the server will not be able to respond to legitimate user. This is the essence of TCP SYN flooding attack.

Theoretically, half-open or embryonic connections are known as TCP SYN-RECV state which you can observe in a server under TCP SYN flooding attack as shown in Figure 3.

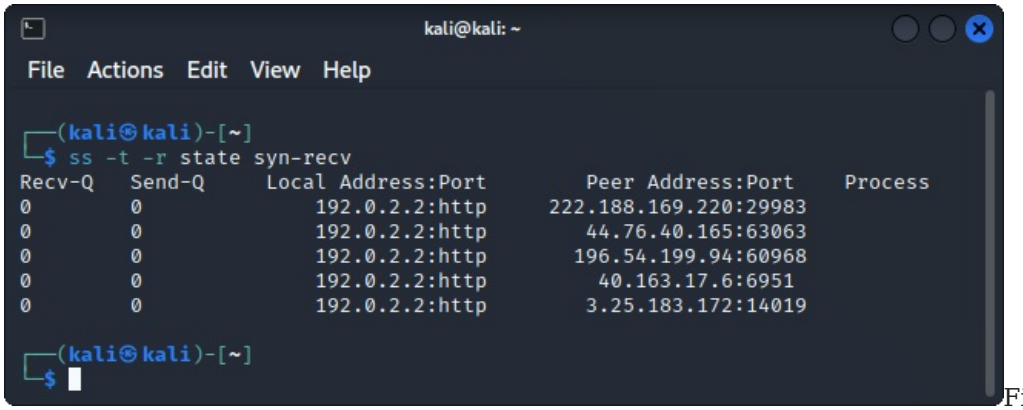

Figure 3

Defending against TCP SYN Flooding Attack

Clearly, ACL is unable to defend against TCP SYN flooding attack. We need capability to limit embryonic connections which fortunately is provided by many modern day firewalls including the Cisco Firepower/ASA firewalls.

Specifically, Firepower/ASA firewalls implement TCP Intercept feature which will be activated when the embryonic connection limit is crossed. This limit can be configured using MPF commands as shown:

```
ciscoasa(config)# access-list accesslist_name extended permit ...
ciscoasa(config)# class-map class_map_name
ciscoasa(config-cmap)# match access-list accesslist_name

ciscoasa(config)# policy-map policy_map_name
ciscoasa(config-pmap)# class class_map_name
ciscoasa(config-pmap-c)# set connection embryonic-conn-max limit
ciscoasa(config-pmap-c)# set connection per-client-embryonic-max limit
```

where *limit* can be between 0 to 2000000 inclusive depending on server's capacity, and the default is 0 which is unlimited connections.

(Remember to configure service-policy to apply the policy-map onto the required interface of the firewall.)

In essence, TCP Intercept acts as a proxy and generates a SYN-ACK spoofing itself as the server to the initiator's SYN request as shown in Figure 4.

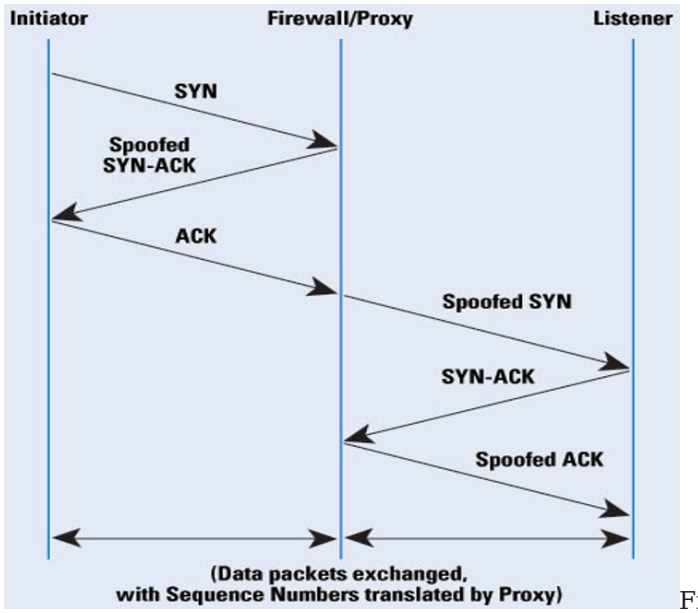

Figure 4

If ACK is received, the original SYN request is not an attack. TCP Intercept will then allow the connection request to go through by sending a SYN spoofing itself as the initiator to the server, and also a corresponding spoofed ACK to complete the connection establishment. Thereafter, TCP Intercept will continue to act as a proxy for the connection between the initiator and the server until it is closed.

If ACK is not received, the original SYN request is likely an attack but will not affect the server. In fact, TCP Intercept has transferred TCP SYN flooding attack targeting the server to the firewall itself! Hence, there is a need for the firewall to defend itself against TCP SYN flooding attack which is achieved using TCP SYN cookie technique implemented in Firepower/ASA firewall.

In a typical TCP connection establishment, whenever a client requests with a SYN connection request, a transmission control block (TCB) will be created and maintained by the server with necessary data to identify each connection. This TCB consumes memory space and is the reason why TCP is vulnerable to SYN flooding attack which consumes all the memory of the server, rendering it unavailable to service legitimate client requests, or possibly crashing the server.

In SYN cookie technique, the TCB is cleverly generated and encoded into the 32-bit sequence number (SN) field as the initial sequence number (ISN) of SYN-ACK and hence the firewall is able to discard the TCB without consuming any memory space as shown in Figure 5.

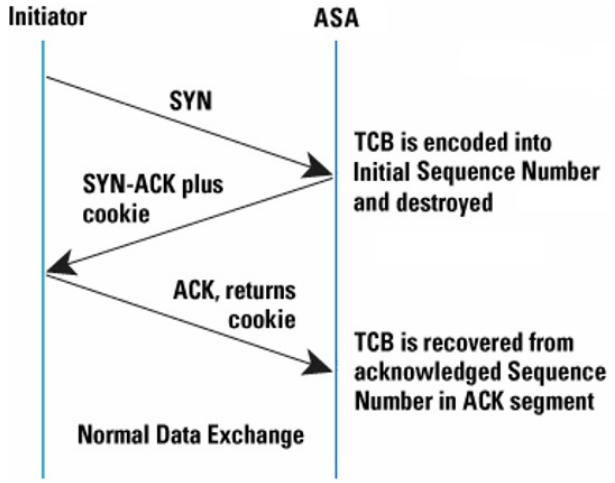

Figure 5

If no ACK is received, the original SYN request is likely an attack but it will not affect the firewall since no memory space is wasted. If ACK is received, the original SYN cookie can be recovered since acknowledgement number (AN) in ACK is equal to SYN-ACK ISN+1. However, the firewall must validate that the SYN cookie is not a fabrication of an attacker before reconstructing the original TCB to continue with the connection request.

The algorithm for generating a SYN cookie varies with different implementations. A feasible example is shown in Figure 6. Notice the pool of local secrets is meant to prevent an attacker from fabricating a valid SYN cookie.

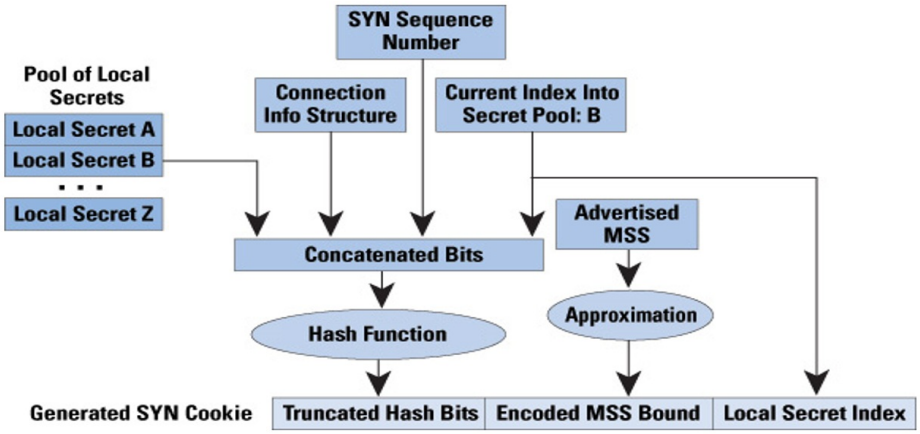

Figure 6

To generate a SYN cookie, a local secret from the pool is randomly selected. It is then concatenated together with a connection information structure containing the IP addresses and port numbers of both client and server which is a part of TCB, the sequence number (SN) contained inside the original SYN request, and the local secret index to form a bit string. This concatenated bit string is then used as input into a cryptographic hash function such as SHA-512 to generate a hash value which is then truncated resulting in truncated hash bits as shown in Figure 6. Since the local secret is part of the input, the truncated hash bits is theoretically a message authentication code which is able to prevent message spoofing attack.

Due to bit-length constraint of the SYN cookie, the maximum segment size (MSS) advertised in the original SYN request needs to be approximated and encoded into shorter MSS bound so that it can be recovered in the ACK if received. Finally, the truncated hash bits, the encoded MSS bound and the local secret index are concatenated together into a 32-bit SYN cookie to be sent as the ISN in the SN field of SYN-ACK.

To validate the SYN cookie in the received ACK, the local secret index is first retrieved from the SYN cookie to locate the local secret in the pool. The connection information structure containing the IP addresses and port numbers of client and server can be recovered from the received ACK as well. Since the SN in ACK is the original SYN SN+1, the SYN SN can also be recovered.

Next, the local secret, the recovered connection information structure, the recovered SYN SN and the local secret index are concatenated together to form a bit string which is then used as input into the same cryptographic hash algorithm to re-generate a truncated hash bits. The re-generated truncated hash bits are then compared with the received truncated hash bits in the SYN cookie in ACK. If they are the same, the ACK is validated since it is computationally infeasible for an attacker to generate a valid SYN cookie without knowing the local secret.

Once the received SYN cookie is validated, the TCB can be re-constructed back from the recovered connection information structure and the encoded MSS bound. Validated connection request can now continue.

However, a word of caution. Even though a firewall can be useful to defend against TCP SYN flooding attack, it has limitation because running TCP Intercept and computing SYN cookie are CPU intensive. If the firewall is being bombarded with an influx of traffic, it can become the choke point and succumb to the attack.

Last but not least, it can be useful to enable threat detection statistics for attacks intercepted by TCP Intercept:

```
ciscoasa(config)# threat-detection statistics tcp-intercept
```

To show the monitored results, enter:

```
ciscoasa(config)# show threat-detection statistics top tcp-intercept
```

A sample of the monitored results is shown in Figure 7.

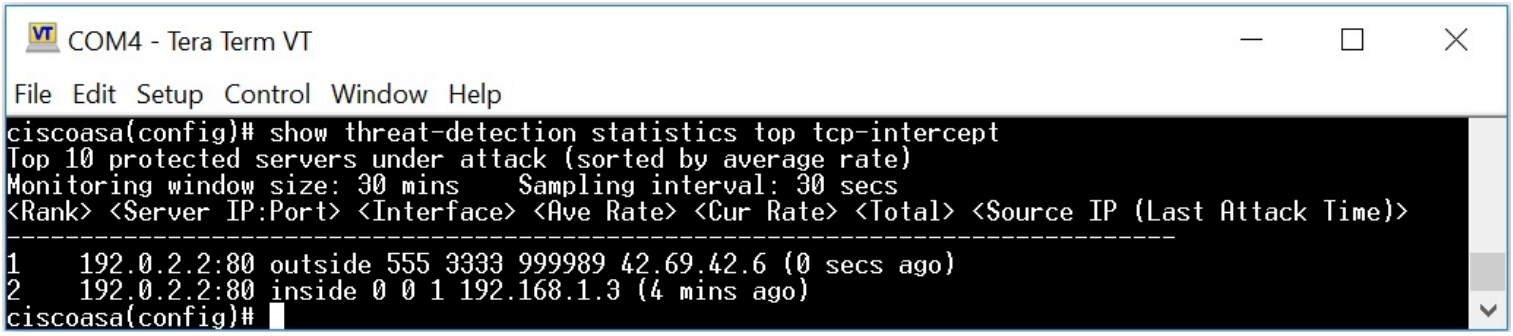

Figure 7